Jacob Beal · CS 6050 · Graduate Algorithm Project 2

Knuth–Plass optimum-fit line breaking

Source: D. E. Knuth & M. F. Plass, "Breaking Paragraphs into Lines," *Software: Practice and Experience* 11(11), 1981, translated from its pseudocode. Code: `knuth_plass/knuth_plass.py`. Theory (p. 1148): O(B·W) for B legal breakpoints, W words per line.

Each line of a paragraph is charged demerits for how far its spaces must stretch or shrink. Each DP entry is the fewest total demerits to break at b with a line of fitness class c. It stays "active" until a line starting there can no longer fit, so each breakpoint examines about W entries.

What I learned

- Line breaking sounds like NP-complete bin packing, but lines hold *contiguous* text, so it reduces to a shortest path through the breakpoints. The 2^B possible settings shrink to a short active list.
- The cheapest way to reach a break can lose: a costlier one can avoid a tight line next to a loose one, so the table is indexed by fitness class. Dropping classes changed just 40 of 148,000 random paragraphs, so a unit test pins one.
- Passing tests can mean little. I broke my code 10 ways on purpose and my first tests missed two. Comparing with the paper's figures also caught me using the wrong column width (421 units, not 390).
- Timing needs a quiet machine. With Xcode compiling, my first run had case 4 about 4× too slow.

How I created the test cases

`knuth_plass/generate_tests.py` (seeded) writes 10 paragraphs whose word count doubles each case, from 2,048 to 1,048,576 words (2,353 to 1,216,218 breakpoints), sampled from the paper's Grimm paragraph to stay realistic. All use a 500-unit column (W ≈ 12.3) and the paper's settings. Expected demerits come from a backward DP that shares only the letter widths with my code. A case passes if its demerits match and rescoring its breaks agrees. Times are the minimum of at least 3 runs, garbage collector off, on an Apple M1 Max with Python 3.12. Unit tests reproduce all 21 line breaks in its Figures 3 and 4(c) and match exhaustive search on thousands of paragraphs. The paper does not print its full font table, so some letter widths are estimates, and total demerits differ from its tables by 9% and 16%.

Runtime vs. theory

With W fixed the bound predicts linear growth in B. In Figure 1 (left) the log–log slope is **1.007** over the five largest cases and 1.017 over all ten, against 1.079 for B log B. Each doubling multiplied the time by 1.96–2.15. The smallest cases sit about 10% low. Part of that is startup: feasible breaks are sparse on a paragraph's opening lines (p. 1163), so the first 250 words average 11.0 active entries per breakpoint against 18.3 afterward. The rest is each entry costing 289 ns in case 1 and 310 ns in case 10, likely cache misses as the node list grows. To test W, I reran case 5 in wider columns (right). The slope is **1.063** once W reaches 45 but 1.201 overall. That is the constant hidden in the O: active entries relative to W rise from 1.5 to about 3.5 and level off, because long lines let each breakpoint keep more of its up to 4 fitness-class entries. With γ = 0, which makes classes irrelevant, that ratio stays at 1.2–1.3 and the slope is 0.932. I believe the implementation matches O(B·W).

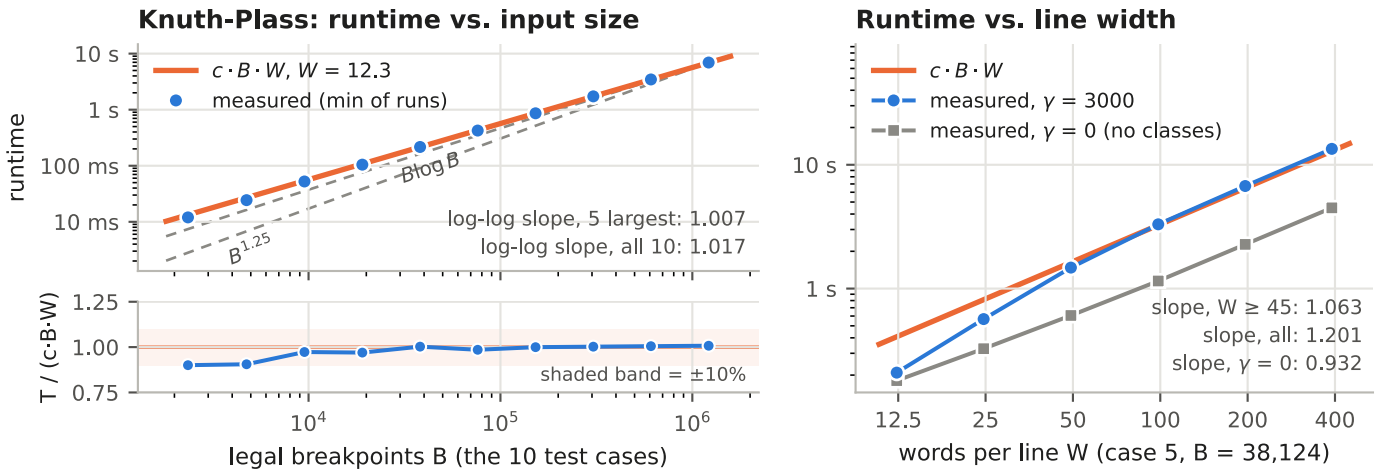


Figure 1. Left: runtime vs. B (log–log) with the fitted c·B·W line (orange) and other growth rates through the largest point (gray, dashed), and below it runtime divided by the fit. Right: runtime vs. W with B fixed, c fitted where W ≥ 45.